NVIDIA Jetson Orin Nano 8GB

Installation guide: boot to NVMe, local + cloud models, voice, Home Assistant, and the OpenClaw agent harness - secured, outbound-only, with human-in-the-loop approval.

Key decisions captured in this build

Version 1.0 - generated 2026-06-14. Pin versions in versions.env; treat this as living documentation.

1. Overview & Architecture

This guide builds an always-on home assistant on a Jetson Orin Nano 8GB with a 500GB NVMe SSD. It serves a frequently-updated multimodal model behind an OpenAI-compatible endpoint and layers the OpenClaw agent harness on top, with strong security defaults.

Components

- llama.cpp (llama-server): local OpenAI-compatible inference for a 3-4B quantized vision model.
- GitHub Copilot provider (optional): cloud models (Claude Opus) to offload the Jetson for heavy reasoning.
- Open WebUI: browser chat interface.
- Wyoming Whisper (STT) + Piper (TTS): voice, integrated with Home Assistant Assist.
- OpenClaw: agentic harness (skills, browser, messaging) running sandboxed.
- WireGuard + VPS: outbound-only admin access and WhatsApp webhook relay.

Architecture (data flow)

```
WhatsApp Cloud API --> VPS (webhook + WireGuard server)
                          |  (relayed over WireGuard tunnel)
                          v
  Jetson Orin Nano 8GB (outbound-only; no inbound router ports)
    - llama-server  :8080  -> OpenAI /v1 (local model)
    - Open WebUI    :3000
    - Whisper :10300  Piper :10200  -> Home Assistant Assist
    - OpenClaw Gateway (sandboxed)
        primary  = github-copilot/claude-opus-4.8  (cloud)
        fallback = llamacpp/<local model>          (offline)
```

Design principles

- Outbound-only: the device initiates all connections; no inbound ports on the home network.
- Approval gates: high-impact actions (orders, payments) require explicit human confirmation.
- Least privilege: the agent holds no payment/order credentials; sandboxed tool execution.
- Version pinning: reproducible deployment via versions.env; avoid 'latest' in production.

2. Hardware Checklist & Memory Budget

Hardware

- Jetson Orin Nano 8GB Developer Kit (Super-capable; JetPack 6.2).
- NVMe M.2 SSD 500GB (PCIe Gen3) installed in the M.2 Key-M slot.
- microSD card (>=64GB) used only for bootstrap.
- Active cooling (fan) - required for sustained inference.
- 12V DC barrel power supply that can sustain MAXN SUPER (do not rely on weak USB-C PD).
- Ethernet (recommended) for a stable headless server.

Memory budget (8GB unified - plan carefully)

Component	Approx. RAM
OS (headless)	~1.5 GB
Local model (3-4B Q4)	3-4 GB
Whisper (base)	0.5-1 GB
Piper (TTS)	<0.2 GB
Open WebUI	~0.3 GB
OpenClaw (Node)	0.3-0.5 GB
Chromium (browser, on-demand)	0.5-1.5 GB

[!] 8GB is the hard constraint

Everythin
8-16GB s
browser-a

3. Phase 1 - Bootstrap to NVMe

The Jetson boots via a Tegra bootloader in QSPI flash on the module, so Windows imaging tools cannot make an SSD bootable on their own. Use the SD card as throwaway bootstrap media, then clone the OS to the NVMe and remove the SD. No x86 Linux host required.

Steps

1) Flash the JetPack 6.2 SD image on Windows with balenaEtcher. 2) Boot once from SD and finish first-boot setup. 3) Update firmware (QSPI) and enable NVMe boot from the Jetson itself:

```
sudo apt update && sudo apt full-upgrade -y   # updates nvidia-l4t-bootloader
sudo reboot
```

4) Clone rootfs SD -> NVMe with the JetsonHacks scripts (run on the Jetson):

```
git clone https://github.com/jetsonhacks/rootOnNVMe.git
cd rootOnNVMe && ./copy-rootfs-ssd.sh && ./setup-service.sh
```

5) Set NVMe first in the UEFI boot order if needed. 6) Shut down, remove the SD, and boot - the system now runs entirely from the NVMe.

> **Note**
>
> Dev kits t
> boot work
> Linux hos

4. Phase 2 - System Configuration

Run on the device after booting from NVMe. See scripts/01-system-config.sh and scripts/02-swap.sh in the repo.

Power, cooling, headless

```
sudo nvpmodel -m 0        # MAXN SUPER
sudo jetson_clocks        # max clocks
sudo systemctl set-default multi-user.target   # disable desktop GUI (frees RAM)
```

Base packages + Node 24 (for OpenClaw)

```
sudo apt install -y build-essential git cmake curl ca-certificates
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs
```

Swap on NVMe (OOM safety net)

```
./scripts/02-swap.sh 12     # creates /mnt/nvme/swapfile, swappiness=10
```

- Static IP or DHCP reservation for a stable headless server.
- SSH is enabled by default on JetPack; manage everything remotely.

5. Phase 3 - Docker & jetson-containers

JetPack ships Docker with the NVIDIA container runtime. Prefer prebuilt Jetson images (dustynv / jetson-containers) over building CUDA from source.

```
docker info | grep -i runtime          # confirm 'nvidia' runtime
git clone https://github.com/dusty-nv/jetson-containers
bash jetson-containers/install.sh
```

Tip

Keep all r

6. Phase 4 - Local Inference (llama.cpp)

llama-server provides an OpenAI-compatible /v1 API for the local multimodal model (privacy/offline fallback). For vision you need the model GGUF and its matching --mmproj projector.

Model choice (balance)

- Gemma 3 4B (Q4_K_M) + mmproj, or Qwen2.5-VL 3B (Q4) + mmproj.
- Download GGUF from a trusted source; verify checksums; pin the filename.
- Store under /mnt/nvme/models.

Run (see compose/docker-compose.yml)

```
llama-server -m /models/gemma-3-4b-it-Q4_K_M.gguf \
  --mmproj /models/gemma-3-4b-mmproj.gguf \
  --host 0.0.0.0 --port 8080 -ngl 99 --ctx-size 4096

curl http://127.0.0.1:8080/v1/models   # health check
```

7. Phase 5 - Open WebUI

A browser chat UI pointed at the local endpoint. Brought up by docker-compose.

```
# environment (compose):
OPENAI_API_BASE_URL=http://llamacpp:8080/v1
OPENAI_API_KEY=sk-no-key-required
# UI on http://<jetson-ip>:3000
```

Security

Bind UI/e
LAN/inter

8. Phase 6 - Voice (Whisper + Piper)

Wyoming services provide speech-to-text and text-to-speech that Home Assistant Assist can discover.

```
  wyoming-whisper  --model base  --language nl   # :10300
  wyoming-piper    --voice nl_NL-mls-medium      # :10200
```

- Load Whisper base/tiny and on-demand to conserve the 8GB budget.
- Both run as containers in docker-compose.yml.

9. Phase 7 - Home Assistant Assist

- Add Wyoming integrations in HA pointing at <jetson-ip>:10300 (Whisper) and :10200 (Piper).
- Create an Assist pipeline: Whisper (STT) -> Conversation agent (LLM) -> Piper (TTS).
- For the LLM, use the Extended OpenAI Conversation custom integration (HACS) so you can set a custom base_url to http://<jetson-ip>:8080/v1.

Approval boundary

HA voice
Keep HA

10. Phase 8 - Connectivity (Outbound-only)

No inbound ports on the home network. The Jetson dials out; interaction returns over the established connections.

Admin: WireGuard

- Run a WG client on the Jetson that dials a WG server on your VPS.
- You reach the Jetson over the private tunnel; the home router opens no ports.
- Keep PersistentKeepalive=25 on the Jetson side. See wireguard/ in the repo.

Agent channel: WhatsApp Cloud API

- The official Cloud API delivers inbound messages via a webhook (inbound by nature).
- Host the webhook receiver on the VPS, verify the Meta signature, and relay to the Jetson over WireGuard.
- Result: the Jetson / home network opens no inbound ports.

11. Phase 9 - OpenClaw Agent Harness

OpenClaw (github.com/openclaw/openclaw, MIT) is a self-hosted personal assistant that bridges chat apps to LLMs with skills, browser control and shell access. Microsoft's 'Scout' is built on OpenClaw. Runtime: Node 24.

Install (pin a version; do not blind-curl)

```
npm install -g openclaw@latest
openclaw onboard --install-daemon     # installs the Gateway systemd service
openclaw gateway status
```

Security configuration (NOT the defaults)

[!] Default = full host access for the main session

You must
require pa

```
# config/openclaw/config.json5 (excerpt)
agents.defaults.sandbox = { mode: 'non-main' }   # Docker sandbox
channels.whatsapp.dmPolicy = 'pairing'           # approve senders explicitly
# approve a paired sender:
openclaw pairing approve whatsapp <code>
```

- Give OpenClaw NO payment/order credentials; browser profile without logged-in webshops.
- High-impact actions behind explicit human approval (OpenClaw approvals).
- Read the official 'Gateway exposure runbook' before any remote exposure.
- ARM64: Node + Playwright/Chromium work; watch for x86-assuming skills.

12. Phase 10 - Optional: GitHub Copilot Cloud Offload

Offload heavy reasoning to cloud models via your GitHub Copilot subscription, keeping the local model as an offline/private fallback. This relaxes the 8GB pressure. OpenClaw supports a github-copilot/* provider natively.

Install the Copilot SDK harness plugin

```
openclaw plugins install @openclaw/copilot   # ~260MB incl. CLI binary
# verify the linux-arm64 Copilot CLI binary loads on the Jetson:
openclaw doctor
```

Preferred model = latest Claude Opus

```
// config/openclaw/config.json5 (excerpt)
agents.defaults.model = {
  primary: 'github-copilot/claude-opus-4.8',   // confirm id: openclaw models list
  fallbacks: ['llamacpp/gemma-3-4b']
}
models: {
  'github-copilot/claude-opus-4.8': { agentRuntime: { id: 'copilot' } },
  aliases: { 'mijn-default': 'github-copilot/claude-opus-4.8' }
}
```

Choosing your model (three levels)

- Config: agents.defaults.model.primary + fallbacks (persistent default).
- CLI: openclaw models list / set, plus aliases.
- In chat: /model picker (per-session, pinned).

13. Security Model & Approval Gates

Capability tiers

- Read-only / informational: may run automatically.
- Reversible actions: notify the user.
- High-consequence / irreversible (orders, payments, external messages): require explicit human approval.

Controls in this build

- Outbound-only networking; no inbound router ports (WireGuard + VPS relay).
- OpenClaw sandboxed (Docker), browser/cron denied by default, DM pairing on.
- No payment/order credentials available to the agent (it can only draft; you execute).
- Secrets in .env (gitignored); never committed. WireGuard private keys generated on-device.
- Audit: keep OpenClaw logs; run 'openclaw doctor' to surface risky policies.

Golden rule

Agents m
enforces t

14. Verification & Benchmarking

```
lsblk                              # root on nvme0n1p1
nvpmodel -q                        # MAXN SUPER
curl http://127.0.0.1:8080/v1/models
docker compose ps                  # all services up
openclaw gateway status
# quick local inference latency:
time curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model":"local","messages":[{"role":"user","content":"hi"}]}'
```

15. Maintenance & Updates

- Update models: download a new GGUF, update the filename in compose + versions.env, restart llamacpp.
- Update OpenClaw: npm update -g openclaw (then 'openclaw doctor'); pin the version in versions.env.
- Update containers: bump tags in versions.env deliberately, then docker compose pull && up -d.
- Keep the OS current: sudo apt update && sudo apt full-upgrade (includes firmware).
- Commit every change to the deployment repo so the running state is captured.

16. Troubleshooting

Symptom	Fix
NVMe won't boot after clone	Update firmware (apt full-upgrade), check UEFI boot order
OOM / killed processes	Use 3-4B model, add swap, run Chromium on-demand
pip install torch broken	Use Jetson-specific wheels / jetson-containers, not default pip
Copilot runtime missing	openclaw plugins install @openclaw/copilot; openclaw doctor
WhatsApp not receiving	Check VPS webhook + Meta signature + WireGuard tunnel
Solcast/secrets empty	Secrets in .env, never committed; check env_file mapping

Appendix - Version Pin Table

Mirror of versions.env. Update deliberately; verify on-device before bumping.

Item	Pinned value
JetPack	6.2
L4T	r36.4
llama.cpp image	dustynv/llama_cpp:r36.4.0
Open WebUI	ghcr.io/open-webui/open-webui:main
Wyoming Whisper	rhasspy/wyoming-whisper
Wyoming Piper	rhasspy/wyoming-piper
Node	24
OpenClaw	pin once chosen
Cloud model	github-copilot/claude-opus-4.8
Local model	llamacpp/gemma-3-4b (Q4)